

Acme Corp Q1 2026 Infrastructure Report

Prepared by: Platform Team | Date: April 2, 2026 | Classification: Internal

Executive Summary

Q1 2026 was a strong quarter for Acme Corp infrastructure. Overall system uptime reached 99.94%, exceeding our 99.9% SLA target. We processed 847 million API requests across all services, a 34% increase over Q4 2025. Average API response time improved from 180ms to 142ms (p50) and from 520ms to 380ms (p95), driven by the caching improvements deployed in Sprint 29-31.

Key achievements this quarter:

- Migrated authentication from session-based to JWT (ADR-0042), reducing auth service load by 60%
- Deployed read-through caching for user profiles, achieving 87% cache hit ratio (up from 34%)
- Completed the Kubernetes cluster upgrade from 1.28 to 1.29 with zero downtime
- Reduced monthly infrastructure spend by 12% through right-sizing ElastiCache and RDS instances

We had two notable incidents: a 23-minute API gateway outage on February 12 (human error during config deployment) and a 45-minute degradation of the notification service on March 8 (RabbitMQ memory pressure). Both are detailed in the Incident Review section.

Uptime and Availability

Service uptime for Q1 2026 (target: 99.9% per service):

- User Service: 99.97% (13 minutes total downtime)
- API Gateway: 99.95% (22 minutes total downtime, see INC-2026-0212)
- Order Service: 99.99% (4 minutes total downtime)
- Notification Service: 99.87% (57 minutes total downtime, see INC-2026-0308)
- Web Application: 99.98% (9 minutes total downtime)

Aggregate platform uptime: 99.94%

The notification service was the only service below our 99.9% target. The root cause was a memory leak in the RabbitMQ consumer that has since been patched (PR #4521). We have added memory pressure alerts at 70% and 85% thresholds to catch similar issues earlier.

Traffic and Performance

API request volume by month:

- January: 261 million requests (avg 97K req/hour)
- February: 278 million requests (avg 114K req/hour)
- March: 308 million requests (avg 115K req/hour)
- Q1 Total: 847 million requests

Peak traffic: March 15 at 14:00 UTC -- 312K requests/hour during the product launch event. All services remained healthy at peak load. The API gateway autoscaled from 4 to 8 pods.

Response time improvements (comparing Jan Week 1 to Mar Week 4):

User Service:

- p50: 95ms to 62ms (35% improvement, driven by cache)
- p95: 340ms to 195ms (43% improvement)
- p99: 890ms to 520ms (42% improvement)

API Gateway:

- p50: 12ms to 11ms (minimal change, already fast)
- p95: 45ms to 38ms
- p99: 120ms to 95ms

Order Service:

- p50: 180ms to 165ms (8% improvement)
- p95: 520ms to 480ms (8% improvement)
- p99: 1200ms to 1050ms (13% improvement)

The user service saw the largest improvement due to the caching layer deployed in Sprint 29-31.

Infrastructure Cost

Monthly AWS spend (all services):

- January: \$14,200
- February: \$13,800
- March: \$12,500 (after right-sizing in Sprint 31)
- Q1 Total: \$40,500

Compared to Q4 2025 total of \$46,200, this represents a 12.3% reduction.

Cost breakdown by category:

- EKS (compute): \$6,800/month (54%)
- RDS (database): \$3,200/month (26%)
- ElastiCache (Redis): \$180/month -> \$90/month after downsize (1%)
- S3 + CloudFront: \$350/month (3%)

- Data transfer: \$1,200/month (10%)
- Other (monitoring, secrets, DNS): \$680/month (5%)

The biggest savings came from right-sizing the RDS instance (db.r6g.xlarge to db.r6g.large, saving \$900/month) after the caching layer reduced database load.

Incident Reviews

INC-2026-0212: API Gateway Outage (February 12)

Duration: 23 minutes (14:07 - 14:30 UTC)

Severity: SEV-1 (full API unavailability)

Impact: All API requests returned 503 errors. Web app and mobile app non-functional.

Root cause: An engineer deployed a Kong configuration update that included an invalid upstream reference. The configuration passed local validation but the target service name had a typo ("user-servce" instead of "user-service"). Kong accepted the config but could not route requests.

Timeline:

- 14:07: Config deployed via ArgoCD
- 14:08: Monitoring alerts fire (error rate > 50%)
- 14:10: On-call engineer paged
- 14:15: On-call identifies bad Kong config in the deploy diff
- 14:22: Rollback initiated via ArgoCD
- 14:30: Service fully restored

Action items (all completed):

1. Add integration test that validates upstream references against actual service names
2. Add canary step to Kong config deployments
3. Update runbook with Kong-specific rollback procedure

INC-2026-0308: Notification Service Degradation (March 8)

Duration: 45 minutes (09:15 - 10:00 UTC)

Severity: SEV-2 (single service degradation)

Impact: Email and push notifications delayed by up to 45 minutes. No data loss.

Root cause: A memory leak in the RabbitMQ consumer caused notification service pods to hit their 512MB memory limit and get OOMKilled by Kubernetes. The leak was introduced in PR #4401 (Feb 28) when a database connection pool was not properly closed in an error path.

Action items (all completed):

1. Fix the connection pool leak (PR #4521, hotfixed same day)
2. Add memory pressure alerts at 70% and 85% of pod limit
3. Add connection pool metrics to Grafana dashboard
4. Establish policy: all database connections must use context managers

Capacity Planning and Q2 Outlook

Current capacity utilization (March averages):

- EKS cluster CPU: 42% (6 nodes, m6i.xlarge)
- EKS cluster memory: 58%
- RDS PostgreSQL CPU: 28% (down from 45% pre-cache)
- RDS storage: 120GB / 500GB (24%)
- ElastiCache Redis memory: 8MB / 6.38GB (<1%)
- RabbitMQ: 2% memory, 5% disk

We have significant headroom on all resources. The March 15 peak (312K req/hour) used approximately 65% of cluster capacity.

Projected growth for Q2:

- User base: 142 -> ~500 users (enterprise sales pipeline)
- API traffic: ~300M/month -> ~500M/month
- Database size: 120GB -> ~200GB

This growth is well within current capacity. No infrastructure changes needed for Q2.

Q2 Infrastructure Priorities

1. Order service performance optimization
 - Database query optimization (N+1 queries identified in 3 endpoints)
 - Connection pooling with PgBouncer
 - Target: p95 < 300ms (currently 480ms)
2. Observability improvements
 - Distributed tracing with OpenTelemetry
 - Service dependency map in Grafana
 - SLO dashboards with burn rate alerts
3. Disaster recovery testing
 - Conduct first full DR test (failover to us-west-2)
 - Document and automate RDS failover procedure
 - Test backup restoration from S3
4. Security hardening
 - Enable AWS GuardDuty for threat detection
 - Implement Kubernetes network policies
 - Rotate all service account credentials
5. Developer experience
 - Reduce CI pipeline time from 8 minutes to under 5 minutes
 - Add preview environments for PRs (using Argo Rollouts)

- Standardize local dev setup with devcontainers

Report prepared by Alex Nowak and Sarah Chen, Platform Team.

For questions, reach out in #team-platform on Slack.